Six-unit orbit table and rolling accumulator: evidence report

Date: 2026-10-19
Scope: the 18-position secp256k1 nearest-digit table, six-unit orbit encoding, rolling point accumulator, and its frozen direct-versus-rolling comparison. This report is an experiment result, not a general scalar-multiplication performance claim.

Question and method

Can the exact 12,211-norm nearest-digit set and terminal ball be represented by six-unit orbit seeds, and can the selected unit actions be accumulated with at most two aggregate rotations while preserving the output point?

The integer screen checks all 12,211 residues, the six-unit orbit representatives, the 699 terminal seeds, and the generated residue-to-code table. The finite Lean covariance proof and the rolling identity are linked from the experiment README. The affine group replay and hash-pinned Rust overlays compare the produced points against independent binary scalar multiplication. The frozen case-0 receipt also records the operation counts for the direct and rolling evaluators.

Curve and subgroup: secp256k1, `EC1P256Csecp256k1ha8dc9f115a15` (the identity in `crypto:docs/curves/regis`
$y^2=x^3+7 \bmod p$, with `p = FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFC2F`.
The prime subgroup has order `n = FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141`, cofactor 1, and generator coordinates `Gx = 79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F` `Gy = 483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8`. The recoding operates on the GLV coefficient lattice with the experiment's fixed τ eigenvalue. No new curve or isogeny is introduced.

Findings

- **Derived and exhaustively screened:** the six-unit action preserves the digit norm, has residue multiplier 12,100 modulo 12,211, and maps every selected digit to its canonical seed and unit/sign code. The screen records 2,036 digit seeds and 699 terminal seeds.
- **Independently replayed:** affine and native outputs match independent binary scalar multiplication on the reported frozen and fresh deterministic inputs. The rolling case-0 implementation uses two aggregate rotations instead of ten affine `x` constant multiplications, with the same 18 nontrivial mixed additions.
- **Measured, but unqualified for latency:** RunPod Tier 1 job `9cf2881670f8411a8850c19ab6aa0525`, allocation `thyvmdqmzsz5ik` (RunPod SECURE `cpu3c`, EU-RO-1, 8 vCPU/16 GB), Rust 1.93.1, five frozen scalar cases, eight alternating repetitions per case. The recorded host is x86_64 Ubuntu 24.04.4 under Docker, reported CPU model AMD EPYC 9655 96-Core Processor. The child benchmark was pinned to CPU 13 from the exposed affinity set `{2,13,40,74,98,109,136,170}`; one NUMA node was reported. CPU quota was unlimited and host exclusivity and memory policy were unverified; physical memory type/speed/channel data were unavailable and remain unknown. All 240 arms and 120 pairs completed, outputs verified, and artifact hashes were stable. Five of 40 adjacent A/A comparisons fell outside the frozen [0.80, 1.25] band. The descriptive median direct/rolling online wall-time ratio was 0.9548; `qualified_latency_ratio` is null. It is not a qualified speedup or latency ranking.
- **Memory:** the committed native layout reports 2,448,196 retained bytes versus 7,193,344 bytes for the sign-only nearest-digit implementation. The 2,390,208-byte figure is point payload only; the native count includes code and table metadata.

The plot uses the five per-scalar medians and min–max observed ranges from `tier1-analysis.json`. The shaded band is the preregistered A/A acceptance interval, not a confidence interval for the candidate comparison. See the linked raw panel receipts and failed calibration cells in the README; no failed arm is removed.

Visuals and graph audit

The algorithm diagram shows the seed-code and accumulation path; its editable source is orbit-rolling.dot. The editable graph source regenerates the quantitative plot from the committed analysis JSON. The PDF report embeds both vector figures.

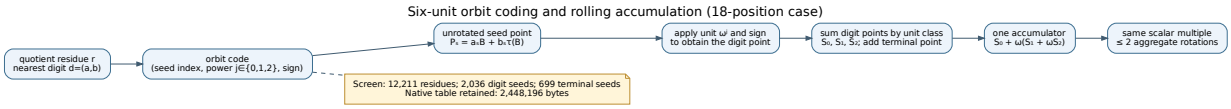


Figure 1: Six-unit orbit coding and rolling accumulation

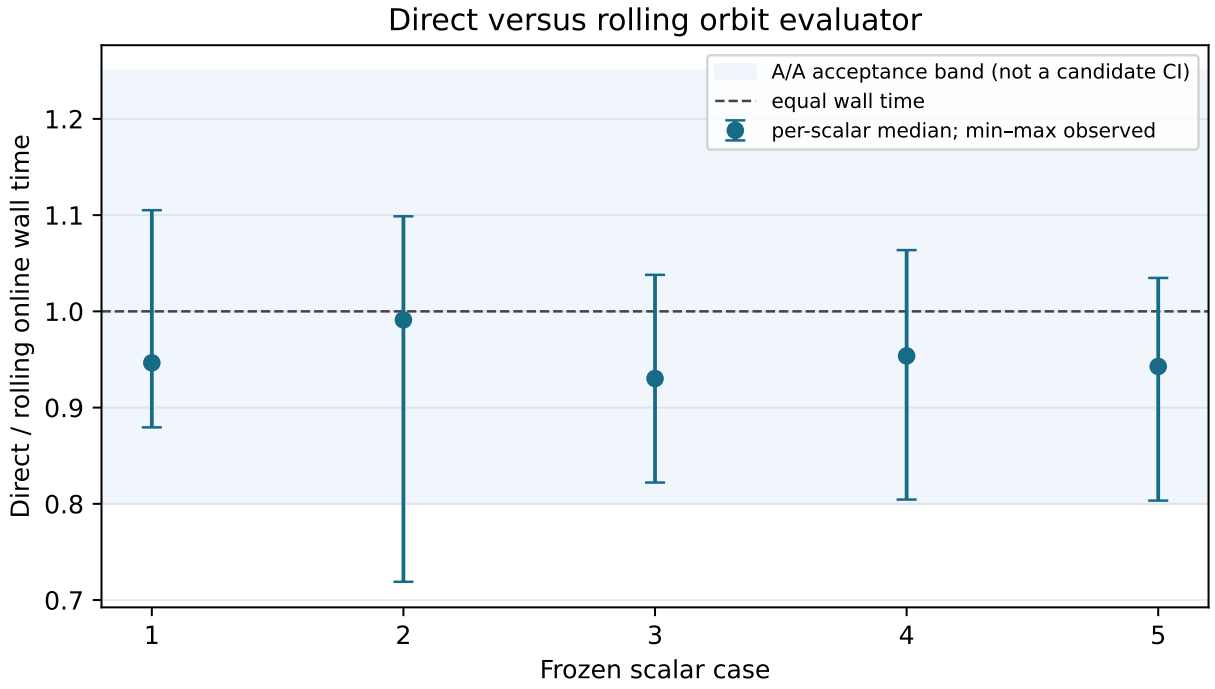


Figure 2: Direct-versus-rolling panel; 5 failed A/A controls and unqualified latency

Checked `docs/BENCHMARKS.md`, `docs/perf/PERFORMANCE_INDEX.md`, `docs/perf/weights.json`, and `experiments/volcano-ic/figures/` (`census.svg/.png`, `ecdlp.svg/.png`, and `tau.png`). The benchmark index covers the C library and suite kernels, while the volcano figures cover index-calculus structure; this report measures a separate Rust fixed-base scalar-table experiment. Its only direct-versus-rolling latency panel failed the declared A/A qualification gate. No canonical benchmark or graph row is affected or changed. The secp256k1 curve record is referenced from the sibling crypto registry; no candidate curve or isogeny route is added here.

Open checks and reproduction

The latency comparison remains unqualified. A longer warmed batch with the same frozen scalars, alternating order, and A/A gate would be needed before ranking direct and rolling latency; that work is not part of this report. Reproduce the existing analysis with `python3`

`experiments/prime-j0-nearest-orbit-12211-20261014/analyze_tier1.py`; it validates the preserved run receipts and does not launch a benchmark.

The formal claims are limited to the integer orbit and rolling identities. The concrete curve action is checked by independent affine and native replay rather than a Lean proof of the secp256k1 point group. See the README's theorem table and retained receipts for exact boundaries.

Source records

- Experiment report and reproduction commands
- Frozen panel analysis
- RunPod allocation and host probe
- Raw runs and pairs
- Integer screen and Lean checks, Lean covariance proof
- Source revision: PR #898 head `efbda14e91dc148a3f98f0f7c82e3e500f3bc2c5`